



Sistema de Controle Descentralizado de Estoque e FIFO (PWA)

Pacote Completo de Entregas — Unidade Curricular: Arquitetura e Modelagem de Sistemas

Projeto	Sistema de Controle Descentralizado de Estoque e FIFO
Indústria demandante	Cristal Master Indústria e Comércio Ltda (Polímeros)
Integrantes	1 - <i>Christoffer Henrique da Silva Souza</i> 2 - <i>Ferdinando Rainert</i> 3 - <i>Gabriel Angelo Foppa</i>



Preencha os campos entre colchetes com os dados reais da equipe antes de publicar/entregar.



Sumário

1. [Ferramentas, Estrutura do Projeto e Planejamento Inicial](#)
2. [Fundamentos de Design de Software: Requisitos, Responsabilidades e Arquitetura em Camadas](#)
3. [Curadoria e Síntese de Evidências: Escolha do Banco de Dados](#)
4. [Technology Radar](#)
5. [Modelagem Comportamental UML](#)
6. [Paradigmas de Modelagem e Programação](#)
7. [Mapa de Diagramas UML](#)
8. [Matriz de Rastreabilidade](#)

1. Ferramentas, Estrutura do Projeto e Planejamento Inicial

1.1 Contexto

O almoxarifado da Cristal Master enfrenta divergência entre estoque físico e estoque registrado, quebra do método FIFO e uso de fichas de papel, pois o ERP está restrito a computadores desktop. O projeto propõe um PWA (Progressive Web App) que leve o registro de movimentações para o dispositivo móvel do operador, com leitura de código via câmera e validação obrigatória de FIFO.

1.2 Seleção das Ferramentas

Categoria	Ferramenta	Justificativa
Documentação	Google Drive (Docs)	Edição colaborativa em tempo real, versionamento automático de rascunhos e centralização dos artefatos do projeto em um único repositório acessível à equipe.
Comunicação	WhatsApp / Discord	Comunicação rápida entre os integrantes para alinhamento diário de tarefas e decisões de arquitetura.
Versionamento	GitHub	Controle de versão do backend (Spring Boot) e do frontend (React), histórico de commits, branches por funcionalidade e revisão de código via Pull Requests.
Prototipação	Figma	Criação dos protótipos de tela otimizados para uso mobile (dedo/luva), validando a usabilidade antes da implementação.

Categoria	Ferramenta	Justificativa
Modelagem	Draw.io (diagrams.net)	Criação dos diagramas UML (classes, casos de uso, sequência, estados) e do diagrama de arquitetura em camadas.
Gestão de Tarefas	Trello / GitHub Projects	Organização do backlog em formato Kanban, com rastreabilidade de quem está fazendo o quê.
Contêinerização	Docker + Docker Compose	Padronização do ambiente de backend, banco de dados e frontend entre os membros da equipe e o ambiente de homologação.
Testes de API	Insomnia / Postman	Validação manual dos endpoints REST antes da integração com o frontend.

Por que essa seleção é superior à de um projeto genérico: diferente de um conjunto de ferramentas escolhido apenas por familiaridade, cada item aqui resolve uma dor específica do MVP — por exemplo, o Docker Compose não é opcional neste projeto porque o ambiente precisa incluir banco de dados, backend e (futuramente) um proxy HTTPS, exigido pela própria API de câmera do navegador (`getUserMedia` só funciona em contexto seguro).

1.3 Estrutura de Pastas do Projeto

```

├── 01_Documentacao
|
|   ├── Requisitos
|   |
|   |   ├── Requisitos_Funcionais_RF
|   |   |
|   |   ├── Requisitos_Nao_Funcionais_RNF
|   |   |
|   |   └── Tabela_de_Requisitos
|   |
|   └── Regras_de_Negocio_RN

```

- | | | — RN01_Controle_de_Acesso
- | | | — RN02_Cadastro_de_Produtos_e_Lotes
- | | | — RN03_Validacao_FIFO
- | | | — RN04_Movimentacao_de_Estoque
- | | | — RN05_Excecao_e_Justificativa_FIFO
- | | — Casos_de_Uso
- | | | — Levantamento_dos_Casos_de_Uso
- | | | | — Lista_de_Casos_de_Uso
- | | | | — Diagrama_de_Casos_de_Uso
- | | | — Especificacao_dos_Casos_de_Uso
- | | | — UC01_Autenticar_Usuario
- | | | — UC02_Cadastrar_Produto
- | | | — UC03_Cadastrar_Lote
- | | | — UC04_Ler_Codigo_do_Lote
- | | | — UC05_Registrar_Saida_com_Validacao_FIFO
- | | | — UC06_Registrar_Entrada
- | | | — UC07_Registrar_Excecao_FIFO
- | | | — UC08_Consultar_Saldo_Disponivel
- | | — Diagramas
- | | | — Diagrama_de_Classes
- | | | — Diagrama_de_Casos_de_Uso
- | | | — Diagrama_de_Sequencia

- | |— Diagrama_de_Estados
- | └─ Diagrama_de_Camadas
- |
- |— 02_Prototipos
- | |— Telas_do_Sistema
- | └─ Arquivos_Figma
- |
- |— 03_Desenvolvimento
- | |— backend
- | | └─ src/main/java/com/estoque
- | | |— controller
- | | |— service
- | | | └─ FifoValidationService.java
- | | |— repository
- | | |— entity
- | | |— dto
- | | |— mapper
- | | |— exception
- | | |— security
- | | └─ config
- | └─ frontend

- | └─ src
- | └─ components
- | | └─ ScannerCamera
- | | └─ FifoAlertModal
- | └─ pages
- | └─ services
- | └─ hooks
- | └─ types
- | └─ contexts
- | └─ layouts
- | └─ utils
- |
- | └─ 04_Testes
- | └─ Plano_de_Testes
- | └─ Casos_de_Testes
- | └─ Relatorio_de_Erros
- |
- | └─ 05_Gerenciamento_do_Projeto
- | └─ Cronograma
- | └─ Kanban
- | └─ Divisao_de_Tarefas
- | └─ Controle_de_Entregas

|

└─ 06_Imagens_e_Materiais

└─ Logos

└─ Capturas_de_Tela

└─ Referencias

1.4 Quadro de Tarefas (Kanban Inicial)

Coluna	Cartão (Tarefa)	Responsável
A Fazer	Revisar e validar RF/RNF junto à demanda da indústria	<i>[Integrante 1]</i>
A Fazer	Documentar as regras de negócio (RN), com destaque para a regra FIFO	<i>[Integrante 1] / [Integrante 4]</i>
A Fazer	Levantar e especificar os casos de uso (UC01–UC08)	<i>[Integrante 4]</i>
A Fazer	Criar os diagramas UML (Classes, Casos de Uso, Sequência, Estados)	<i>[Integrante 3]</i>
A Fazer	Prototipar as telas críticas: leitura de código e bloqueio FIFO	<i>[Integrante 3]</i>
A Fazer	Modelar o banco de dados (Produto, Lote, Movimentação, Usuário)	<i>[Integrante 2]</i>
A Fazer	Implementar o <code>FifoValidationService</code> e sua cobertura de testes	<i>[Integrante 2]</i>
A Fazer	Configurar Docker Compose (backend + MySQL) e pipeline no GitHub	<i>[Integrante 2]</i>

Coluna	Cartão (Tarefa)	Responsável
A Fazer	Organizar a documentação no Google Drive	[Integrante 1]
A Fazer	Criar cronograma de 42h e acompanhar entregas	[Integrante 1]
A Fazer	Elaborar plano de testes, com foco no cenário de violação de FIFO	[Integrante 4]
A Fazer	Revisar integração entre documentação, protótipo e código	Todos os integrantes

1.5 Primeiros Requisitos Registrados

ID	Nome do Requisito	Descrição	Tipo	Prioridade	Origem	Critério de Aceitação
RF01	Autenticação de Usuário	O sistema deve permitir login do operador via usuário e senha.	Funcional	Alta	Cliente	O operador deve autenticar-se e receber um token válido para operar o sistema.
RF04	Leitura de Código	O sistema deve permitir a leitura de código de barras/QR via câmera do dispositivo.	Funcional	Alta	Cliente	O código lido deve identificar corretamente o produto e o lote correspondente.

ID	Nome do Requisito	Descrição	Tipo	Prioridade	Origem	Critério de Aceitação
RF06	Validação FIFO	O sistema deve validar se o lote informado é o mais antigo disponível antes de autorizar a saída.	Funcional	Alta	Cliente	Uma tentativa de retirada fora de ordem deve ser identificad a antes da confirmaçã o da baixa.
RNF05	Autenticação Stateless	A autenticação deve ser stateless, via JWT.	Não Funcional	Alta	Equipe de Desenvolvimento	Nenhuma sessão deve ser mantida em memória no servidor entre requisições.

1.6 Identificação do Grupo

Integrante	Papel no Projeto
<i>[Integrante 1]</i>	Gerente de Projeto e Analista de Requisitos — planejamento, organização do backlog, levantamento e validação dos RF/RNF junto à demanda da indústria.
<i>[Integrante 2]</i>	Analista de Arquitetura e Backend — definição da arquitetura em camadas, modelagem do banco de dados, implementação da regra de negócio FIFO.
<i>[Integrante 3]</i>	Analista de Modelagem e Frontend/UX — diagramas UML, protótipos das telas mobile e integração da leitura de código via câmera.

Integrante	Papel no Projeto
[Integrante 4]	Analista de Documentação e Qualidade — especificação dos casos de uso, regras de negócio, plano de testes e revisão dos artefatos.

2. Fundamentos de Design de Software: Requisitos, Responsabilidades e Arquitetura em Camadas

2.1 Requisitos Funcionais Detalhados

RF01 – Autenticação de Usuário Descrição: O sistema deve permitir login de operador via usuário e senha, retornando um token JWT. Tipo: Funcional · Prioridade: Alta · Interessado: Almoxarifado / TI Critério de Aceitação: O operador deve autenticar-se com credenciais válidas e receber um token JWT; credenciais inválidas devem ser rejeitadas com mensagem clara.

RF04 – Leitura de Código via Câmera Descrição: O sistema deve permitir a leitura de código de barras/QR via câmera do dispositivo para identificar produto, lote ou posição de armazenagem. Tipo: Funcional · Prioridade: Alta · Interessado: Operador de Almoxarifado Critério de Aceitação: Ao apontar a câmera para um código válido, o sistema deve reconhecer produto e lote em menos de 2 segundos, sem etapas intermediárias.

RF06 – Validação da Regra FIFO Descrição: O sistema deve validar, a cada tentativa de saída, se existe lote do mesmo produto com data de entrada anterior e saldo disponível. Tipo: Funcional · Prioridade: Alta (núcleo do sistema) · Interessado: Almoxarifado / Qualidade Critério de Aceitação: Toda tentativa de retirar um lote que não seja o mais antigo disponível deve ser identificada antes da confirmação da baixa.

RF07 – Bloqueio Visual de Violação FIFO Descrição: O sistema deve bloquear visualmente a operação (modal de alerta) quando o operador tentar consumir lote fora da ordem FIFO, indicando o lote correto. Tipo: Funcional · Prioridade: Alta · Interessado: Almoxarifado Critério de Aceitação: O modal deve exibir claramente o número do lote correto e impedir a confirmação da baixa até a exceção ser tratada.

RF08 – Exceção Justificada ao FIFO Descrição: O sistema deve permitir que o operador registre uma exceção à regra FIFO mediante justificativa obrigatória (ex.: lote avariado). Tipo: Funcional · Prioridade: Média · Interessado: Supervisor de Almoxarifado Critério de Aceitação: A exceção só é aceita com justificativa preenchida e fica marcada (`violou_fifo = true`) para auditoria posterior.

(RF02, RF03, RF05, RF09–RF12 seguem o mesmo padrão de detalhamento e estão descritos de forma resumida na tabela da Seção 1.5 e no documento de arquitetura já produzido pela equipe.)

2.2 Identificação das Responsabilidades Necessárias

Autenticação e Controle de Usuários

- Realizar login e emissão de token JWT.
- Controlar permissões conforme o perfil (Operador / Supervisor).
- Registrar o usuário responsável por cada ação sensível.

Cadastro de Produtos e Lotes

- Registrar produtos com código e código de barras.
- Registrar lotes vinculados a um produto, com data de entrada, validade e posição de armazenagem.
- Garantir unicidade de número de lote por produto.

Leitura de Código (Scanner)

- Capturar o código via câmera do dispositivo.
- Decodificar e resolver o produto/lote correspondente.
- Lidar com falha de leitura (código ilegível, sem câmera disponível) de forma explícita.

Validação e Orientação FIFO

- Ordenar os lotes de um produto pela data de entrada.
- Determinar se o lote informado é o mais antigo com saldo disponível.
- Sugerir o lote correto quando houver violação.
- Registrar exceções justificadas para auditoria.

Movimentação de Estoque

- Registrar entradas e saídas.
- Atualizar o saldo disponível do lote em tempo real.
- Vincular a movimentação a uma OP (Ordem de Produção).

Auditoria e Relatórios

- Manter histórico completo de movimentações.
- Disponibilizar consulta de saldo em tempo real.
- Sinalizar movimentações que violaram o FIFO, mesmo quando justificadas.

2.3 Diagrama de Camadas

O sistema adota uma arquitetura em quatro camadas, isolando a interface móvel das regras de negócio e da persistência dos dados.

graph TD

A[Operador de Almoxarifado] --> B

subgraph B["Camada de Apresentação — PWA React + TypeScript"]

B1[Tela de Login]

B2[ScannerCamera - Leitura de Código]

B3[FifoAlertModal - Bloqueio FIFO]

B4[Consulta de Saldo]

B5[Registro de Entrada/Saída]

end

B --> C

subgraph C["Camada de Aplicação — Spring Boot Controllers"]

C1[Endpoints REST]

C2[Tradução DTO ↔ Domínio]

C3[Tratamento Central de Erros]

end

C --> D

```
subgraph D["Camada de Domínio — Services"]
```

```
    D1[FifoValidationService]
```

```
    D2[MovimentacaoService]
```

```
    D3[LoteService / ProdutoService]
```

```
    D4[AuthService]
```

```
end
```

```
D --> E
```

```
subgraph E["Camada de Infraestrutura"]
```

```
    E1[(MySQL via JPA/Hibernate)]
```

```
    E2[Flyway - versionamento de schema]
```

```
    E3[Docker]
```

```
    E4[Logs e Auditoria]
```

```
end
```

2.4 Componentes com Responsabilidades

Componente	Responsabilidade
AuthService	Autenticação, emissão de JWT e controle de permissões por perfil.
ProdutoService	Cadastro, consulta e desativação de produtos.
LoteService	Cadastro de lotes, controle de saldo disponível e posição de armazenagem.
MovimentacaoService	Registro de entradas/saídas e vínculo com OP.

Componente	Responsabilidade
FifoValidationService	Ordenação de lotes por data de entrada e verificação da regra FIFO a cada saída.
Repository Layer	Comunicação com o MySQL via Spring Data JPA.

2.5 Relação Requisitos ↔ Componentes

Requisito	Componentes Responsáveis
RF01 – Autenticação	AuthService + Repository Layer
RF04 – Leitura de Código	ScannerCamera (frontend) + ProdutoService + LoteService
RF06 – Validação FIFO	FifoValidationService + LoteService
RF07 – Bloqueio Visual	FifoAlertModal (frontend) + FifoValidationService (backend)
RF08 – Exceção Justificada	MovimentacaoService + FifoValidationService
RF09 – Saldo em Tempo Real	LoteService + Repository Layer

2.6 Configurações Necessárias Identificadas

Configuração	Descrição	Requisitos Relacionados
Banco de Dados	Armazenamento de produtos, lotes, movimentações e usuários.	RF02, RF03, RF06, RF09
Autenticação e Autorização	Controle de login e permissões por perfil (Operador/Supervisor).	RF01, RNF05
Contexto Seguro (HTTPS)	Necessário para a API de câmera do navegador funcionar em produção.	RF04
Logs de Auditoria	Registro de violações de FIFO e exceções justificadas.	RF08, RNF07

Configuração	Descrição	Requisitos Relacionados
Ambiente Containerizado	Padronização entre desenvolvimento, testes e homologação.	RNF06

2.7 Representação Dinâmica — Fluxo de Saída com Validação FIFO

sequenceDiagram

actor Op as Operador

participant Scan as ScannerCamera

participant Ctrl as Controller

participant Fifo as FifoValidationService

participant Repo as Repository / MySQL

Op->>Scan: 1. Aponta a câmera para o código do lote

Scan->>Ctrl: 2. Envia código lido

Ctrl->>Fifo: 3. Encaminha para validação

Fifo->>Repo: 4. Consulta lotes ordenados por data de entrada

Repo-->>Fifo: 5. Retorna lotes disponíveis

alt Lote correto

Fifo-->>Ctrl: 6a. Aprova a movimentação

Ctrl-->>Op: 7a. Confirma a baixa e atualiza o saldo

else Lote fora de ordem FIFO

Fifo-->>Ctrl: 6b. Retorna violação + lote sugerido

Ctrl-->>Op: 7b. Exibe FifoAlertModal com o lote correto

2.8 Decisões Técnicas Justificadas

Decisão Técnica 01 — Arquitetura em Camadas

Decisão: Adotar quatro camadas — Apresentação, Aplicação, Domínio e Infraestrutura.

Justificativa: Separar a interface móvel (sujeita a mudanças frequentes de UX) da regra de negócio crítica (FIFO), que precisa permanecer estável e testável independentemente da tecnologia de frontend escolhida.

Consequências esperadas:

- Benefícios: manutenção facilitada, testes unitários isolados na camada de domínio, possibilidade de trocar o frontend (ex.: para um app nativo futuro) sem alterar a regra de negócio.
- Possível consequência: aumento inicial do número de classes e pacotes do projeto.

Decisão Técnica 02 — Repository Pattern com Spring Data JPA

Decisão: Utilizar o padrão Repository para isolar o domínio do acesso ao MySQL.

Justificativa: A regra FIFO depende de consultas ordenadas por data de entrada; isolar essa consulta em um Repository permite testar o `FifoValidationService` com mocks, sem depender de um banco real.

Benefícios: menor acoplamento, testes automatizados mais rápidos (JUnit + Mockito), possibilidade de otimizar a consulta (ex.: índice em `data_entrada`) sem alterar a camada de domínio.

Decisão Técnica 03 — PWA em vez de Aplicativo Nativo

Decisão: Entregar a solução como Progressive Web App.

Justificativa: A restrição do projeto veda investimento em nova estrutura física; um PWA elimina a necessidade de distribuição via loja de aplicativos e roda em qualquer smartphone ou coletor com navegador moderno, reduzindo custo e tempo de implantação.

Consequência esperada: menor acesso a APIs nativas de hardware avançadas (ex.: leitores de código dedicados), mitigado pelo uso da API de câmera via `getUserMedia` combinada a uma biblioteca de decodificação em JavaScript.



3. Curadoria e Síntese de Evidências: Escolha do Banco de Dados

3.1 Pergunta de Pesquisa

Qual banco de dados é mais adequado para armazenar produtos, lotes e movimentações de estoque de um sistema industrial que exige forte consistência transacional (a regra FIFO depende de leituras e escritas concorrentes sem inconsistência) e alta frequência de operações de escrita no chão de fábrica?

A pergunta foi escolhida porque a confiabilidade da regra FIFO depende diretamente da capacidade do banco de garantir que duas movimentações concorrentes no mesmo lote não gerem saldo inconsistente.

3.2 Palavras-chave Utilizadas

- MySQL
- PostgreSQL
- MongoDB
- banco de dados relacional
- controle de concorrência (locking)
- transações ACID
- MySQL AND transactions
- PostgreSQL AND row locking
- MongoDB AND transactions
- banco de dados para controle de estoque
- integridade referencial

3.3 Fontes Analisadas

Fonte A — MySQL Tipo: Fonte primária/técnica (documentação oficial). Relação com o projeto: O MySQL oferece transações ACID e bloqueio de linha (`SELECT ... FOR UPDATE`), recurso essencial para impedir que duas requisições concorrentes de baixa no mesmo lote gerem saldo negativo ou duplicado. Relevância: Alta — atende diretamente à necessidade de consistência do saldo por lote.

Fonte B — PostgreSQL Tipo: Fonte primária/técnica. Relação com o projeto: O PostgreSQL também oferece transações ACID robustas, com suporte avançado a constraints e triggers, que poderiam encapsular parte da regra FIFO diretamente no banco. Relevância: Alta — alternativa tecnicamente equivalente ao MySQL para este cenário.

Fonte C — MongoDB Tipo: Fonte primária/técnica. Relação com o projeto: O MongoDB oferece transações multi-documento desde suas versões mais recentes, mas seu modelo orientado a documentos é menos natural para o forte relacionamento Produto → Lote → Movimentação exigido pelo domínio. Relevância: Moderada — viável tecnicamente, mas exige modelagem adicional para replicar garantias que um banco relacional oferece nativamente.

3.4 Comparação das Fontes

Fonte	Modelo	Principais Características	Aplicação no Projeto
MySQL	Relacional	Transações ACID, bloqueio de linha, ampla adoção em ambientes Spring Boot/JPA	Alta — forte candidato, já adotado pela equipe
PostgreSQL	Relacional	Transações ACID, constraints avançadas, triggers	Alta — alternativa equivalente
MongoDB	Orientado a documentos	Flexibilidade de schema, transações multi-documento	Moderada — exigiria modelagem adicional

3.5 Avaliação das Fontes — Framework A²ER²

Fonte	Autoridade	Atualidade	Evidência	Relevância	Rastreabilidade	Avaliação Geral
MySQL — Documentação Oficial	Muito alta	Alta	Muito alta	Muito alta	Muito alta	Muito alta
PostgreSQL — Documentação Oficial	Muito alta	Alta	Muito alta	Muito alta	Muito alta	Muito alta

Fonte	Autoridade	Atualidade	Evidência	Relevância	Rastreabilidade	Avaliação Geral
MongoDB — Documentação Oficial	Muito alta	Alta	Muito alta	Alta	Muito alta	Muito alta

3.6 Evidências, Convergências e Divergências

Evidências: MySQL e PostgreSQL documentam suporte nativo a bloqueio de linha e transações ACID; MongoDB documenta suporte a transações multi-documento, mas recomenda modelagem orientada a agregados para cenários fortemente relacionais.

Convergências: MySQL e PostgreSQL compartilham o modelo relacional, adequado a dados estruturados e interdependentes como Produto → Lote → Movimentação.

Divergências: MySQL possui integração mais direta com o ecossistema Spring Boot/Spring Data JPA já adotado pela equipe; PostgreSQL oferece recursos mais avançados de constraints, porém introduziria uma curva de aprendizado adicional sem ganho relevante para o escopo do MVP; MongoDB exigiria repensar o modelo de dados para um formato de documentos.

3.7 Síntese e Impacto sobre o Projeto

Síntese: As três tecnologias são capazes de armazenar os dados do sistema. A decisão, no entanto, deve priorizar a garantia de consistência transacional no controle de saldo por lote — o ponto mais sensível do sistema, pois é ali que reside o risco de dois operadores baixarem o mesmo lote simultaneamente. MySQL e PostgreSQL atendem esse requisito de forma equivalente; a escolha final por MySQL se apoia na maturidade da integração com Spring Data JPA/Hibernate e na familiaridade da equipe, reduzindo risco de atraso no MVP de 42h.

Impacto sobre o projeto:

- **RF06 (Validação FIFO):** o MySQL sustentará a consulta ordenada por `data_entrada` e o bloqueio de linha (`SELECT ... FOR UPDATE`) na tabela `LOTE`, evitando condição de corrida entre duas baixas simultâneas.
- **RF09 (Saldo em tempo real):** a atualização do campo `quantidade_disponivel` ocorrerá dentro da mesma transação da movimentação, garantindo que o saldo exibido nunca fique defasado.
- **RNF08 (Versionamento de schema):** o Flyway fica responsável por versionar e migrar o schema MySQL ao longo da evolução do projeto.

4. Technology Radar

#	Tecnologia	Problema que Resolve	RF/RNF Relacionado	Maturidade	Decisão
1	PWA + Service Worker	Levar o registro de estoque para o dispositivo móvel sem custo de loja de aplicativos	RF04, RNF01, RNF04	Alta	ADOTAR
2	Leitura de código via câmera (WebRTC + ZXing-js/QuaggaJS)	Identificar produto/lote sem digitação manual no chão de fábrica	RF04	Alta	ADOTAR
3	Docker + Docker Compose	Padronizar ambiente entre backend, banco e frontend	RNF06	Alta	ADOTAR
4	Arquitetura Orientada a Eventos (para futura integração com o ERP corporativo)	Desacoplar o novo sistema do ERP legado, evitando acoplamento direto e permitindo sincronização	Evolução (fora do MVP)	Alta	EXPERIMENTAR

#	Tecnologia	Problema que Resolve	RF/RNF Relacionado	Maturidade	Decisão
		assíncrona futura			
5	IA Generativa aplicada ao desenvolvimento (geração de componentes de UI, esquemas JPA, validações de formulário)	Acelerar a construção de componentes repetitivos do frontend e do backend	Onde a IA atua (escopo do projeto)	Em evolução rápida	AVALIAR
6	Edge Computing / sensores IoT em pallets	Automatizar fisicamente a leitura de posição dos pallets	—	Madura em IoT industrial, mas cara para este caso	EVITAR

Detalhamento — item 4 (Arquitetura Orientada a Eventos): Tecnologia analisada: um barramento de eventos (ex.: RabbitMQ ou um serviço gerenciado equivalente) para publicar eventos como **MovimentacaoRegistrada** ou **FifoViolado**. Principais benefícios: desacopla o módulo de estoque de uma futura integração com o ERP corporativo, permitindo que o ERP consuma o evento quando estiver pronto, sem exigir chamada síncrona. Principais riscos: consistência eventual entre o novo sistema e o ERP; necessidade de desenhar consumidores idempotentes. Decisão da equipe: **EXPERIMENTAR** — não faz parte do MVP de 42h, mas deve ser validado em um spike técnico antes da fase de integração com o ERP.

Detalhamento — item 6 (Edge Computing / IoT): Tecnologia analisada: sensores de posição em pallets/porta-pallets para automatizar a leitura de FIFO fisicamente. Por que evitar: a restrição de "zero investimento em estrutura física" definida pela indústria inviabiliza essa tecnologia neste projeto; toda a inteligência de priorização deve residir no software, não no hardware do almoxarifado.

Resumo do Radar:

- PWA + Service Worker: **ADOTAR**
- Leitura de código via câmera: **ADOTAR**

- Docker + Docker Compose: **ADOTAR**
 - Arquitetura Orientada a Eventos: **EXPERIMENTAR**
 - IA Generativa no desenvolvimento: **AVALIAR**
 - Edge Computing / IoT: **EVITAR**
-
-

5. Modelagem Comportamental UML

5.1 Escopo do Sistema

O sistema é responsável pelo gerenciamento de produtos, lotes, movimentações de estoque e pela validação obrigatória da regra FIFO no chão de fábrica. Para esta entrega, o foco da modelagem comportamental está nos processos de:

- autenticação do operador;
- cadastro de produtos e lotes;
- leitura de código via câmera;
- validação FIFO e bloqueio visual;
- registro de exceção justificada ao FIFO;
- registro de entrada e saída de estoque;
- consulta de saldo disponível.

5.2 Atores do Sistema

Operador de Almoxarifado — realiza a leitura de código, registra entradas e saídas, e trata bloqueios de FIFO no seu dia a dia.

Supervisor de Almoxarifado — autoriza exceções justificadas ao FIFO e consulta relatórios de auditoria.

Sistema de Autenticação — serviço interno responsável por validar credenciais e emitir o token JWT.

ERP Corporativo (*ator externo, fora do MVP*) — sistema legado que futuramente poderá consumir eventos de movimentação de estoque.

5.3 Diagrama de Casos de Uso — Relacionamentos

«include»: Registrar Saída → Validar FIFO

«include»: Registrar Saída → Atualizar Saldo do Lote

«extend»: Registrar Saída → Registrar Exceção Justificada ao FIFO

Descrição dos relacionamentos:

- **«include» — Registrar Saída / Validar FIFO:** toda saída exige, obrigatoriamente, a verificação de qual lote deveria ser retirado primeiro.
- **«include» — Registrar Saída / Atualizar Saldo do Lote:** confirmar uma saída sempre implica atualizar o saldo disponível do lote correspondente.
- **«extend» — Registrar Saída / Registrar Exceção Justificada ao FIFO:** comportamento opcional. Ponto de Extensão: ocorre apenas SE a validação FIFO identificar uma violação e o operador (ou supervisor) optar por registrar uma justificativa em vez de cancelar a operação.

5.4 Especificação Textual — UC05: Registrar Saída de Estoque com Validação FIFO

Ator Principal: Operador de Almoxarifado **Atores Secundários:** Sistema de Validação FIFO

Pré-condições:

- Operador autenticado.
- Produto e lote identificados via leitura de código.
- Existe ao menos um lote do produto com saldo disponível.

Fluxo Principal:

1. O Operador lê o código do lote a ser retirado.
2. O Sistema identifica o produto e o lote correspondente.
3. O Sistema consulta os lotes do mesmo produto ordenados por data de entrada.
4. O Sistema verifica se o lote lido é o mais antigo com saldo disponível.
5. O Sistema aprova a movimentação.
6. O Operador informa a quantidade a ser retirada.
7. O Sistema valida se há saldo suficiente no lote.
8. O Sistema registra a movimentação de saída.
9. O Sistema atualiza o saldo disponível do lote.

10. O Sistema confirma a operação ao Operador.

Fluxos de Exceção:

E4a — Lote fora de ordem FIFO Ocorre no passo 4. O Sistema identifica que o lote lido não é o mais antigo disponível. O Sistema bloqueia visualmente a operação (RF07) e indica o lote correto. O fluxo segue para o UC07 (Registrar Exceção Justificada) ou é cancelado pelo Operador.

E7a — Saldo insuficiente no lote Ocorre no passo 7. O Sistema informa que a quantidade solicitada excede o saldo disponível. O fluxo retorna ao passo 6.

E8a — Falha na persistência Ocorre no passo 8. O Sistema não consegue salvar a movimentação. O Sistema registra a falha no log de incidentes e informa ao Operador que a operação não foi concluída. O caso de uso é encerrado sem confirmar a saída.

Pós-condições:

- Movimentação de saída registrada e vinculada ao usuário responsável.
- Saldo do lote atualizado.
- Caso tenha havido exceção FIFO, o registro fica marcado (`violou_fifo = true`) com a justificativa associada.

5.5 Relação Caso de Uso ↔ Requisito

Caso de Uso	Requisito
UC01 – Autenticar Usuário	RF01
UC02 – Cadastrar Produto	RF02
UC03 – Cadastrar Lote	RF03
UC04 – Ler Código do Lote	RF04
UC05 – Registrar Saída com Validação FIFO	RF06, RF07, RF09
UC06 – Registrar Entrada	RF05
UC07 – Registrar Exceção Justificada ao FIFO	RF08
UC08 – Consultar Saldo Disponível	RF09

6. Paradigmas de Modelagem e Programação

6.1 Requisito Selecionado

RF06/RF07 — Validação e Bloqueio Visual de FIFO Descrição: o sistema deve verificar, a cada saída, se o lote informado é o mais antigo disponível, bloqueando visualmente a operação quando não for.

6.2 Paradigma Predominante

O modelo adotado é a **Orientação a Objetos (OO)**, com a regra de negócio encapsulada em um objeto de domínio dedicado (`FifoValidationService` operando sobre a entidade `Lote`), em vez de espalhada em condicionais dentro do controller.

- **Encapsulamento da regra:** a entidade `Lote` expõe um método de domínio (`ehMaisAntigoDisponivel()`), e o `FifoValidationService` orquestra a consulta e a decisão, sem que o Controller precise conhecer os detalhes da regra.
- **Abertura para extensão (Strategy, se necessário):** caso o negócio precise, no futuro, de critérios alternativos de priorização (ex.: FEFO — First Expired, First Out, baseado na validade em vez da entrada), a lógica de comparação pode ser isolada por trás de uma interface, sem alterar o restante do fluxo.

6.3 Justificativa Técnica

A escolha da Orientação a Objetos, com a regra encapsulada em um serviço de domínio dedicado, justifica-se pela necessidade de manter o núcleo funcional do sistema — a validação FIFO — isolado, testável e imune a duplicação.

Em uma abordagem puramente procedural, a checagem "este lote é o mais antigo?" tenderia a ser reescrita em múltiplos pontos do sistema (no Controller de saída, em uma futura tela de relatórios, em uma futura integração com o ERP), aumentando o risco de divergência entre as implementações e dificultando a manutenção de uma regra que é, por definição, crítica para o negócio.

Ao concentrar a regra em `FifoValidationService`, qualquer novo ponto de entrada do sistema (uma nova tela, uma futura API para o ERP) reaproveita a mesma implementação, e a cobertura de testes automatizados (RNF07) cobre a regra uma única vez, com alta confiança.

6.4 Diagrama Antes e Depois

Visão Procedural (Antes — Regra Espalhada no Controller)

1. O Controller de saída recebe a requisição e, dentro do próprio método, escreve a query SQL para buscar todos os lotes do produto.
2. O Controller compara manualmente as datas de entrada em um laço `for`, decidindo "na mão" qual lote deveria sair primeiro.
3. Se um dia essa mesma checagem for necessária em outra tela (ex.: um relatório de conformidade FIFO), o código é copiado e colado, criando duas fontes de verdade para a mesma regra.
4. Qualquer ajuste futuro na regra (ex.: passar a considerar também a validade do lote) exige encontrar e alterar todas as cópias espalhadas pelo sistema.

Visão Orientada a Objetos (Depois — Regra Encapsulada)

1. O Controller de saída apenas delega a decisão:
`fifoValidationService.validar(loteId)`.
2. O `FifoValidationService` consulta o `LoteRepository` (via Repository Pattern) e aplica a regra de comparação encapsulada na entidade `Lote`.
3. O resultado retorna como um objeto `FifoCheckResult` (válido/inválido + lote sugerido), consumido de forma idêntica por qualquer Controller, relatório ou futura integração.
4. Uma mudança na regra (ex.: adotar FEFO) é feita em um único lugar, sem tocar nos Controllers que a utilizam.

6.5 O Que Mudou e Como

- **Entidades e Estado:** a entidade `Lote` passa a carregar não apenas dados, mas também o comportamento de avaliar sua própria posição na fila FIFO, evitando que essa lógica fique "solta" em uma classe utilitária genérica.
- **Comportamentos:** a responsabilidade de "decidir se pode sair" deixa de pertencer ao Controller e passa a pertencer ao domínio (`FifoValidationService`), que é o único lugar que conhece a regra de negócio por completo.
- **Distribuição de Responsabilidades:** o Controller volta a ter uma única responsabilidade — traduzir a requisição HTTP em uma chamada de serviço e devolver a resposta — enquanto o `FifoValidationService` concentra toda a inteligência de priorização.
- **Dependências e Acoplamento:** o uso do Repository Pattern (Decisão Técnica 02, Seção 2.8) garante que o `FifoValidationService` dependa de uma abstração (`LoteRepository`), e não diretamente do MySQL, permitindo testá-lo com Mockito sem subir um banco real.

7. Mapa de Diagramas UML

Necessidade 1: Estruturação das Entidades de Domínio

- **Informação a ser representada:** as entidades centrais do sistema (Produto, Lote, Movimentação, Usuário), seus atributos e multiplicidades (ex.: um Produto possui zero ou muitos Lotes, mas um Lote pertence a exatamente um Produto).
- **Diagrama UML escolhido:** Diagrama de Classes.
- **Justificativa técnica:** responde à pergunta "quais elementos estruturam o domínio e como se relacionam?", guiando tanto a implementação orientada a objetos quanto a modelagem do banco de dados.
- **Classificação:** Estrutural (visão estática).
- **Requisito relacionado:** RF02, RF03.

classDiagram

```
class Produto {
```

```
    +Long id
```

```
    +String codigo
```

```
    +String codigoBarras
```

```
    +String nome
```

```
    +String descricao
```

```
    +boolean ativo
```

```
}
```

```
class Lote {
```

```
    +Long id
```

```
    +String numeroLote
```

```
+BigDecimal quantidadeInicial  
  
+BigDecimal quantidadeDisponivel  
  
+LocalDate dataEntrada  
  
+LocalDate dataValidade  
  
+String posicaoArmazenagem  
  
+ehMaisAntigoDisponivel() boolean  
}
```

```
class MovimentacaoEstoque {  
  
    +Long id  
  
    +TipoMovimentacao tipo  
  
    +BigDecimal quantidade  
  
    +LocalDateTime dataMovimentacao  
  
    +String opReferencia  
  
    +boolean violouFifo  
  
    +String justificativa  
}
```

```
class Usuario {  
  
    +Long id  
  
    +String nome  
  
    +String login  
  
    +String senhaHash
```

+Perfil perfil

}

Produto "1" --> "0..*" Lote : possui

Lote "1" --> "0..*" MovimentacaoEstoque : registra

Usuario "1" --> "0..*" MovimentacaoEstoque : executa

Necessidade 2: Escopo Funcional e Interação Externa

- **Informação a ser representada:** os atores que interagem com o sistema (Operador, Supervisor, futuramente o ERP) e as funcionalidades acessíveis a cada perfil.
- **Diagrama UML escolhido:** Diagrama de Casos de Uso.
- **Justificativa técnica:** responde a "quem utiliza o sistema e para quê?", delimitando a fronteira do sistema sem expor regras de negócio internas.
- **Classificação:** Comportamental (visão dinâmica de alto nível).
- **Requisito relacionado:** Escopo geral do MVP.

Necessidade 3: Ciclo de Vida do Lote

- **Informação a ser representada:** as transições de status que um Lote sofre ao longo do tempo (**Ativo** → **Parcialmente Consumido** → **Esgotado**), e os eventos que disparam cada transição.
- **Diagrama UML escolhido:** Diagrama de Estados (Máquina de Estados).
- **Justificativa técnica:** responde a "quais estados um lote pode assumir?", modelando restrições como "um lote Esgotado não pode voltar a ser Ativo", o que é central para a confiabilidade da regra FIFO.
- **Classificação:** Comportamental (visão dinâmica).
- **Requisito relacionado:** RF06, RF09.

stateDiagram-v2

[*] --> Ativo: Entrada registrada (RF05)

Ativo --> ParcialmenteConsumido: Saída parcial validada (RF06)

ParcialmenteConsumido --> ParcialmenteConsumido: Nova saída parcial

ParcialmenteConsumido --> Esgotado: Saldo chega a zero

Ativo --> Esgotado: Saída total validada

Esgotado --> [*]

Necessidade 4: Fluxo de Leitura de Código e Validação FIFO em Tempo Real

- **Informação a ser representada:** a ordem cronológica de troca de mensagens entre ScannerCamera (frontend), Controller, FifoValidationService e Repository ao processar uma leitura de código.
 - **Diagrama UML escolhido:** Diagrama de Sequência.
 - **Justificativa técnica:** responde a "em qual ordem os componentes se comunicam?", detalhando os bastidores do fluxo descrito na Seção 2.7 e evidenciando o ponto exato em que a decisão de bloquear ou aprovar é tomada.
 - **Classificação:** Interação (subgrupo comportamental/dinâmico).
 - **Requisito relacionado:** RF04, RF06, RF07.
-
-

8. Matriz de Rastreabilidade

8.1 O Que É a Matriz de Rastreabilidade

A matriz liga cada requisito pedido pela indústria a um artefato concreto do projeto (caso de uso, tela, diagrama). Ela serve para provar duas coisas: que nenhuma exigência da Cristal Master ficou sem solução prevista, e que a equipe não construiu nada que não tenha sido pedido.

8.2 Tabela de Rastreabilidade

ID	O que a indústria pediu	Tipo	Caso de Uso	Tela (Protótipo)	Diagrama	Cobertura	Observações
RF01	Login do operador	RF	UC01	Tela 00 – Login	Diagrama de Sequência	Tudo Certo	—

ID	O que a indústria pediu	Tipo	Caso de Uso	Tela (Protótipo)	Diagrama	Cobertura	Observações
RF02	Cadastro de produtos	RF	UC02	Tela 01 – Produtos	Diagrama de Classes	Tudo Certo	—
RF03	Cadastro de lotes	RF	UC03	Tela 02 – Lotes	Diagrama de Classes	Tudo Certo	—
RF04	Leitura de código via câmera	RF	UC04	Tela 03 – Scanner	Diagrama de Sequência	Tudo Certo	Depende de contexto HTTPS em produção (ver RNF a definir).
RF06	Validar FIFO na saída	RF	UC05	Tela 04 – Saída	Diagrama de Estados + Sequência	Tudo Certo	É o núcleo do sistema.
RF07	Bloquear visualmente violação de FIFO	RF	UC05 (fluxo E4a)	Tela 04 – Saída (Modal)	—	Faltam Peças	O modal está descrito textualmente, mas ainda não foi desenhado no Figma.
RF08	Exceção justificad	RF	UC07	—	—	Faltam Peças	Falta desenhar a tela

ID	O que a indústria pediu	Tipo	Caso de Uso	Tela (Protótipo)	Diagrama	Cobertura	Observações
	a ao FIFO						onde o supervisor autoriza a exceção e registra a justificativa.
RF09	Consulta de saldo em tempo real	RF	UC08	Tela 05 – Saldo	—	Faltam Peças	Falta um diagrama de sequência específico para o cenário de consulta simultânea por dois operadores.
RF10	Vínculo de movimentação a uma OP	RF	UC05 / UC06	Tela 04 – Saída	—	Faltam Peças	O campo <code>op_referencia</code> existe no modelo de dados, mas não há tela dedicada para consultar

ID	O que a indústria pediu	Tipo	Caso de Uso	Tela (Protótipo)	Diagrama	Cobertura	Observações
							OPs em aberto.
RNF05	Autenticação stateless (JWT)	RNF	UC01	Tela 00 – Login	Diagrama de Sequência	Tudo Certo	—
RNF07	Testes automatizados da regra FIFO	RNF	—	—	Decisão Técnica 02 (Repository Pattern)	Tudo Certo	Cobertura prevista via JUnit + Mockito.
RNF09	Senhas em hash	RNF	UC01	—	Regra no AuthService	Faltam Peças	Precisa ser formalizado como regra de negócio (RN) documentada.
—	Ponto de pedido automático (sugerido na Seção 9 do documento original de arquitetura)	Evolução	—	—	—	Peça Inventada / Fora do Escopo	É uma sugestão de evolução, não um requisito confirmado pela indústria — não deve ser implementada no MVP

ID	O que a indústria pediu	Tipo	Caso de Uso	Tela (Protótipo)	Diagrama	Cobertura	Observações
							sem validação formal com o cliente.

8.3 Problemas Encontrados e Como Corrigir

- **O Desejo Parcialmente Atendido (RF07, RF08, RF09, RF10):** a equipe especificou textualmente o comportamento esperado (bloqueio visual, exceção justificada, consulta de saldo, vínculo com OP), mas ainda não produziu o protótipo visual correspondente a cada um. **Como consertar:** priorizar no Figma, ainda nesta semana, as quatro telas listadas como "Faltam Peças", pois RF06/RF07 são o núcleo do MVP e não podem chegar à implementação sem validação visual prévia com o time.
- **A Regra Sem dono Formal (RNF09 — senhas em hash):** o requisito é tecnicamente óbvio e por isso corre o risco de nunca ser formalizado como Regra de Negócio (RN) documentada, ficando apenas na cabeça de quem implementa. **Como consertar:** criar a RN06_Seguranca_de_Credenciais na pasta de Regras de Negócio, deixando explícito o algoritmo de hash escolhido (ex.: BCrypt) e a política de expiração do token JWT.
- **A Peça Inventada (Ponto de Pedido Automático):** esse item aparece no documento de arquitetura como sugestão de evolução, mas não corresponde a um requisito formalmente solicitado pela Cristal Master. **Como consertar:** manter esse item apenas no backlog de evolução (Seção "Sugestões para Evolução"), sem alocar esforço de modelagem ou desenvolvimento a ele até que a indústria confirme o interesse formalmente.

Como Executar o Projeto (referência rápida)

Clonar o repositório

git clone <url-do-repositorio>

cd <nome-do-repositorio>

```
# Subir backend + banco de dados via Docker Compose
```

```
docker compose up -d
```

```
# Rodar o frontend (PWA)
```

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Ajuste este bloco assim que o `docker-compose.yml` e os scripts de inicialização estiverem definidos no repositório.



Licença

Projeto acadêmico desenvolvido para a Unidade Curricular de Arquitetura e Modelagem de Sistemas, em atendimento à demanda da indústria Cristal Master. Uso restrito a fins educacionais, salvo autorização em contrário.